
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We propose FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries data, and our central contribution is to show that pairing an autoregressive factorization with flow matching yields the best of both worlds. Given historical measurements and optional future covariates, we formulate forecasting as sampling from a learned conditional distribution over future trajectories. Concretely, FLOWTIME makes three contributions. First, rather than modeling an entire fixed-horizon future window at once, we decompose the joint distribution of future observations into a sequence of per-step conditional densities under a Markov assumption, and we model each density with a *single shared* flow that transforms a simple base distribution into the next observation, conditioned on past observations and covariates. Second, we learn these transformations within the flow matching (FM) framework, which makes training scalable and simulation-free and, unlike prior FM forecasters, requires no informed or hand-crafted priors, so the method extends directly to high-dimensional multivariate data. Third, we show empirically that this design retains the hallmark strengths of autoregressive models—strong extrapolation beyond the training horizon, a compact model, and well-calibrated uncertainty—while still capturing the complex, multi-modal conditional distributions that transport-based generative models are prized for. We validate all three contributions on classical stochastic dynamical systems and on real-world forecasting benchmarks, where FLOWTIME matches or surpasses strong statistical, diffusion, and flow-matching baselines.

1 INTRODUCTION

Probabilistic timeseries forecasting—extrapolating how system dynamics evolve into the future from observational data—is a core problem in modern machine learning and is central to a wide range of scientific, industrial, and societal disciplines (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024). This paper targets that problem and contributes a forecasting model that is at once scalable to high-dimensional multivariate data, strong at extrapolation, and well-calibrated in its uncertainty.

An emerging trend is to leverage deep generative models to tackle this problem (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021), framing forecasting as sampling from a future probability density conditioned on the past. Most notably, diffusion models and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have recently pushed state-of-the-art performance across multiple forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). Despite this impressive performance, diffusion models typically incur high computational costs during both training and inference—a cost our approach is designed to avoid.

A promising route around that cost is Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022), an emerging paradigm for generative modeling that generalizes and subsumes diffusion models while offering more flexible design choices. Unlike diffusion models, which rely on iterative stochastic denoising steps over a discretized trajectory, FM learns deterministic probability paths that transform arbitrary base distributions into the target distribution directly through continuous normalizing flows. By optimally designing these probability paths, FM circumvents the

handcrafted noise schedules and lengthy ancestral sampling chains inherent to diffusion, enabling more efficient training and sampling. FM is the foundation we build on.

FM has recently been applied to timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024), showing strong empirical results and improved computational efficiency over diffusion models. We identify a specific limitation that these methods share, and it is the gap our contributions close: current approaches learn the conditional distribution of an entire fixed-horizon future window in one shot, conditioned on a fixed-horizon context window. While this makes training and sampling fast, it also yields models that extrapolate poorly beyond the training distribution and produce miscalibrated uncertainty estimates. It further introduces a harder optimization problem, in which the model must learn intricate time-dependent probability paths along the flow dimension. This last issue is highlighted by (Kollovieh et al., 2024), who use informed priors based on Gaussian processes to simplify and speed up optimization—but at the cost of restricting their approach to univariate forecasting.

Our approach. We propose FLOWTIME, a simple alternative that uses FM to learn probabilistic forecasting models that scale to high-dimensional multivariate datasets *without* relying on informed priors. The key idea is a change of factorization: rather than modeling the entire future window simultaneously, FLOWTIME decomposes forecasting into a sequence of per-step conditional distributions, modeling each future time step conditioned on past observations and covariates through a single shared flow that transports a simple base distribution to the next observation’s distribution. This autoregressive structure is what lets FLOWTIME handle variable forecasting horizons, extrapolate beyond the training distribution, and deliver well-calibrated uncertainty, all while simplifying the optimization problem.

Contributions. In summary, this paper makes the following contributions:

- **An autoregressive flow-matching formulation for forecasting.** We recast probabilistic forecasting as an autoregressive product of per-step conditional densities under a Markov assumption, and learn every step with one shared flow (Section 1 and the method that follows). This is, to our knowledge, the first FM forecaster to combine an autoregressive factorization with a shared per-step flow.
- **Prior-free scalability to multivariate data.** Because the per-step densities are simpler than a full joint over the horizon, FLOWTIME trains without the informed Gaussian-process priors that restrict prior FM forecasters to the univariate setting, and thus applies directly to high-dimensional multivariate timeseries.
- **Empirical validation of the autoregressive advantage.** Through a controlled ablation against a non-autoregressive FM baseline on five stochastic dynamical systems, and comparisons against strong statistical, diffusion, and flow-matching baselines on five real-world datasets, we show that FLOWTIME delivers markedly better extrapolation and competitive-to-state-of-the-art accuracy and calibration.

Taken together, these contributions offer a practical and effective solution to probabilistic timeseries forecasting, which we substantiate on both classical dynamical systems and real-world tasks.

2 BACKGROUND: FLOW MATCHING

Before presenting FLOWTIME, we review the flow matching machinery on which our contributions rest. Flow matching (FM) (Lipman et al., 2022) is a paradigm for generative modeling that enables training of continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at an unprecedented scale. CNFs model data transformations as solutions to ordinary differential equations (ODEs), providing invertible mappings with tractable likelihoods.

Similar to CNFs, FM aims to learn a time-dependent diffeomorphic map defined on the data space Ω , called a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transforms a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ such that $\mathcal{X}^1 \sim q$ for some target distribution q . The flow is constructed by solving the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

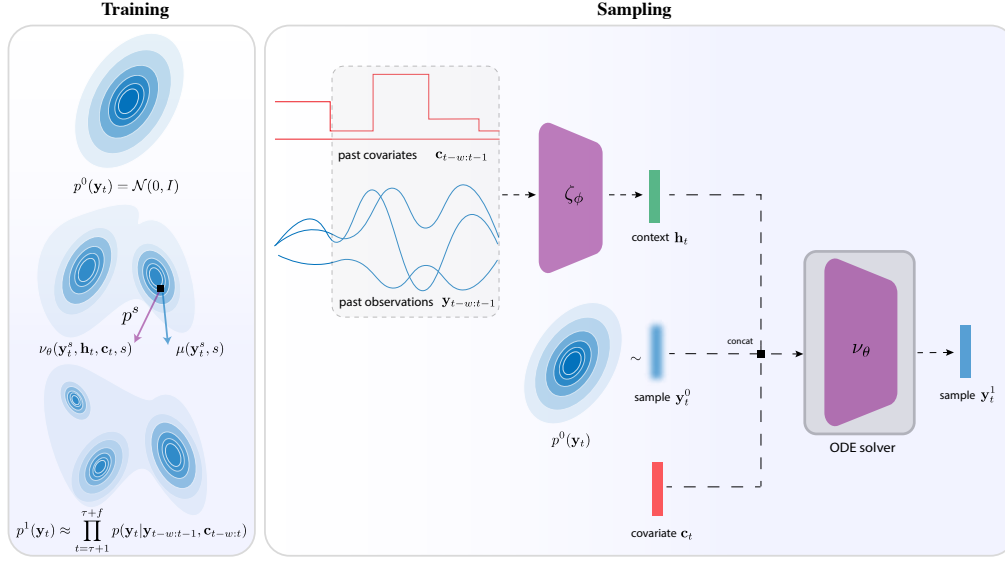


Figure 1: Overview of FLOWTIME, illustrating the shared per-step flow that is our first contribution. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ is constructed between a base distribution p^0 and the target distribution p^1 , generated by a velocity field μ . Training regresses μ with a neural network ν_θ that takes in a sample from the probability path y_t^s at flow step s , a context vector $\mathbf{h}_t$ encoding past observations and covariates, and the current covariate $\mathbf{c}_t$. At inference, forecasting proceeds autoregressively: we sample the base distribution p^0 and integrate the trained velocity field with an ODE solver until $s = 1$ to produce the next observation, then roll the window forward.

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is a vector field defining the velocity of the flow and generating a probability path $(p^s)_{0 \leq s \leq 1}$ where each p^s is a distribution over Ω with $p^0 = p$. The objective is thus to learn a valid vector field μ such that $p^1 = q$.

Unlike CNFs where the vector field is learned via likelihood maximization, requiring solving and differentiating through Eq. (1) during training, FM learns a vector field ν_θ with parameters θ by regressing over vector fields of fixed conditional probability paths. This enables defining a tractable objective function called conditional flow matching (CFM) defined as:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Minimizing this objective allows sampling from q by solving Eq. (1) with the trained ν_θ . By avoiding gradient backpropagation through the ODE solver, FM enables stable and scalable training for high-dimensional generative tasks such as image and video modeling (Esser et al., 2024; Polyak et al., 2024)—precisely the scalability our second contribution transfers to multivariate forecasting. Various probability path designs and conditioning strategies have been proposed, unifying multiple transport-based generative models under this framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023); FLOWTIME inherits this flexibility and applies it per time step.

3 AUTOREGRESSIVE FLOW MATCHING

This section develops the first contribution of the paper: an autoregressive flow-matching formulation of forecasting in which a single shared flow models every per-step conditional density. We first fix notation and the problem setting, then state the autoregressive factorization and spell out why it helps, and finally derive the target probability path, the training objective, and the inference procedure.

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given access to its l past measurements including the current observation, denoted as $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, our objective is to provide probabilistic forecasts of the next f future values denoted as $\mathbf{Y}_f =$

$\{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, potentially conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, our goal is to learn to sample from the conditional distribution $p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$ using a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ with m the number of instances.

At the heart of FLOWTIME, we propose to factorize the conditional distribution autoregressively across future time-steps under a Markov assumption of order w , such that:

$$p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. Our objective thus becomes to learn the conditional distribution in Eq. (3). Departing from the maximum likelihood estimation common for autoregressive models, our contribution is to learn this conditional distribution with the flow matching framework (Lipman et al., 2022)—combining the autoregressive structure with a transport-based generative model.

This autoregressive factorization is the source of FLOWTIME’s advantages, and it directly underwrites the strengths we validate empirically. First, it reduces the learning problem to modeling simpler conditional distributions $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ rather than a complex high-dimensional joint—the very simplification that lets us drop the informed priors of prior work (our second contribution). Second, each time step’s conditional distribution can be trained within a teacher forcing framework, allowing parallel training. Third, the Markov assumption with window size w offers an explicit trade-off between model capacity and computational complexity. Finally, the formulation enables interpretable uncertainty quantification, explicitly modeling how prediction uncertainty propagates through time via the chain of conditional distributions.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

Having stated the factorization, we now instantiate the shared per-step flow that realizes it. For all future time steps $t \in [\tau+1, \tau+f]$, we define a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$, with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$ where $p_{\mathcal{D}}$ is the empirical distribution. This probability path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along s defined via the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a vector field constructing the flow chosen such that the resulting flow satisfy the continuity equation.

To derive a tractable objective, we can model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) shows that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional probability paths $p^s(\mathbf{y}_t | \mathbf{z})$ are equivalent to their respective marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. We define the conditional probability path as:

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \rightarrow 0$. The respective conditional velocity field is then simply $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice is deliberate: it yields straight probability paths from source to target in Euclidean space and closed-form solutions, which is exactly what keeps training and sampling efficient and lets us dispense with the informed priors used by prior FM forecasters. Learning this velocity field lets us sample from the target distribution $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by solving the ODE in Eq. (4) until $s = 1$.

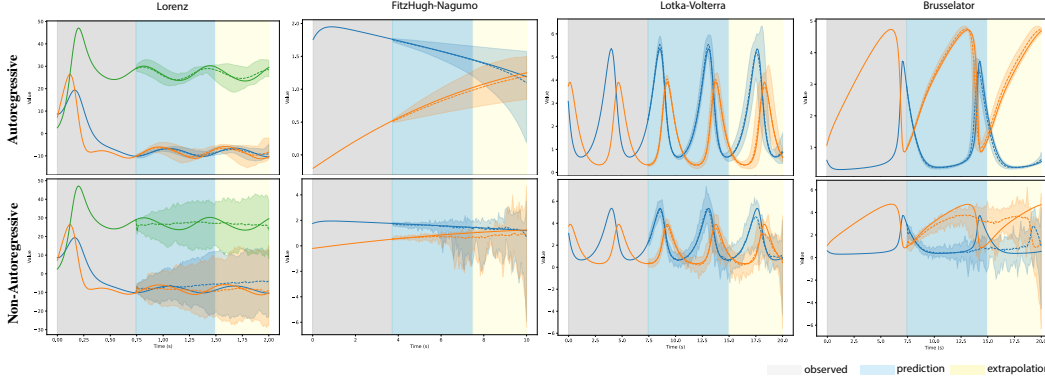


Figure 2: Qualitative evidence for our main claim: autoregressive flow matching (FLOWTIME) versus standard (non-autoregressive) flow matching on four dynamical systems. Solid lines are ground truth; dashed lines are the mean prediction with the 95% confidence interval. Showing both the prediction and extrapolation regimes makes the contribution visible—FLOWTIME tracks the dynamics and keeps its uncertainty calibrated well past the training horizon, where the non-autoregressive baseline degrades.

3.2 TRAINING

Training operationalizes the shared-flow contribution: a single network learns the per-step velocity field for all time steps at once. As in the standard FM setting, we learn this velocity field by regressing it against a neural network ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To do this efficiently in our setting, we define a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . We can then define our learning objective as:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by randomly sampling an observation at time point t , its associated context window, and covariates from $\mathcal{D}$, and jointly updating the parameters ϕ and θ to minimize $\mathcal{L}(\theta, \phi)$ via stochastic gradient descent. This is akin to teacher forcing (Williams & Zipser, 1989): ground-truth past observations are fed into the model during training (rather than its own past predictions), which is what allows parallel training across all time-steps and avoids error accumulation—a practical payoff of the autoregressive formulation. Algorithm 1 summarizes the training procedure for autoregressive flow matching.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
 - 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
 - 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
 - 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
 - 5: Compute target: $\mu(\mathbf{y}_t^s, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
 - 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
 - 7: Update θ, ϕ
 - 8: **end while**
 - 9: **return** ν_θ, ζ_ϕ
-

3.3 INFERENCE

Inference is where the autoregressive structure pays off in flexible horizons and calibrated uncertainty. After training, we produce probabilistic predictions for new observations $\mathbf{Y}_l$ and correspond-

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

```
1: Initialize  $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$ 
2: for  $t = \tau + 1$  to  $\tau + f$  do
3:   Set context:  $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$ , using the last  $w$  entries in  $\mathbf{Y}_f$ 
4:   Sample  $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ 
5:   Solve  $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s)$ ,  $\psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0$ ,  $s \in [0, 1]$ 
6:   Append  $\psi(\mathbf{y}_t, 1)$  to  $\mathbf{Y}_f$ 
7: end for
8: return  $\mathbf{Y}_f$ 
```

ing covariates $\mathbf{C}$ over f future timestamps. Following the sampling procedure in Algorithm 2, predictions are made autoregressively using a rolling-window approach, so the same trained model serves any forecast horizon. Repeating this procedure many times yields empirical quantiles that quantify predictive uncertainty.

4 EXPERIMENTS

Our experiments are organized to test the three contributions directly. Section 4.1 isolates the effect of the autoregressive factorization (contribution 1) through a controlled ablation on five stochastic dynamical systems, with a particular emphasis on extrapolation. The claim of prior-free scalability to multivariate data (contribution 2) is then stressed on five real-world benchmarks, where FLOWTIME is compared against strong statistical, diffusion, and flow-matching baselines (contribution 3). Together these studies establish that FLOWTIME improves extrapolation and delivers competitive-to-state-of-the-art accuracy and calibration.

4.1 DYNAMICAL SYSTEMS

Data To test contribution 1 in a controlled setting, we evaluated FLOWTIME on forecasting five stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka–Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, we generated trajectories by numerically solving the corresponding stochastic differential equations, using randomly sampled initial conditions and Brownian motion realizations. The details of the dynamical systems and the data generation process are provided in Appendix A.1.

Implementation details Reflecting the compact-model claim, these experiments use a deliberately small architecture: ζ_ϕ is a two-layer bi-directional LSTM with 64 hidden units, and ν_θ is an MLP with three hidden layers of size 64. We encode the flow step s with Fourier positional embeddings into a 16-dimensional vector using a fixed set of frequencies, and set the context length w equal to the prediction length. We trained via stochastic gradient descent with an Adam optimizer, a learning rate of 0.003, and a batch size of 128. Training to convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the contribution of the autoregressive factorization, we compare FLOWTIME against a non-autoregressive flow-matching baseline that directly models the joint distribution of future observations conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. Because the two share everything but the factorization, this is a clean ablation that attributes any difference to contribution 1 alone. This non-autoregressive formulation closely resembles the method proposed in (Kollovich et al., 2024). Further details on this baseline are provided in Appendix B.

Evaluation The evaluation protocol is built to expose the extrapolation claim. We partitioned each trajectory into three segments: (1) an observation window that serves as historical input to the model; (2) a prediction window, available only during training as the target; and (3) an extrapolation window that remains entirely unseen during training. The third segment is what tests generalization beyond the training horizon. We report the continuous rank probability score (CRPS) (Winkler et al., 1996)

to measure uncertainty quantification and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to measure predictive accuracy.

Table 1: Ablation supporting contribution 1: autoregressive (FLOWTIME) vs non-autoregressive (Non-AR) factorization, holding everything else fixed, on five stochastic dynamical systems. We report CRPS and NRMSE for both the prediction and extrapolation regimes on the test set; the extrapolation columns are the key test of generalization beyond the training horizon. Values are mean and standard deviation across five random seeds, and the better factorization in each cell is shown in **bold**.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 supports contribution 1: the autoregressive factorization in FLOWTIME substantially improves forecasting across stochastic dynamical systems. **On prediction accuracy**, the AR approach beats the non-AR baseline in NRMSE on all five systems, most dramatically on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658)—reductions of 93% and 95%—and yields better CRPS on four of five systems, the Van der Pol oscillator being the lone exception on uncertainty quantification. **On extrapolation**—the more demanding test and the crux of the claim—the AR advantage in NRMSE holds across all five systems, evidencing superior generalization beyond the training distribution; the Brusselator case is emblematic, with a 98% reduction in extrapolation NRMSE (0.023 vs 1.046). In short, modeling temporal dependencies through autoregressive factorization lets FLOWTIME better capture the underlying dynamics, translating into more accurate predictions and better-calibrated uncertainty, especially in the challenging extrapolation regime.

4.2 REAL-WORLD DATASETS

Data To test the prior-free multivariate scalability of contribution 2, we evaluated FLOWTIME on forecasting multiple univariate and multivariate timeseries of differing frequencies (hourly and daily) from GluonTS (Alexandrov et al., 2020), namely Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). These span up to thousands of dimensions—the high-dimensional regime that prior FM forecasters could not reach without informed priors. We provide further details in Appendix A.2.

Implementation details For ζ_ϕ we use a three-layer bi-directional LSTM with 64 hidden units, and for ν_θ a residual network of five residual blocks built from 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows (Rasul et al., 2021) but with fewer residual blocks—keeping the model compact in line with our stated strengths. The flow step s is encoded with Fourier positional embeddings into a 32-dimensional vector using a fixed set of frequencies, and the same embedding encodes time-dependent covariates $\mathbf{c}$. We trained via stochastic gradient descent with an Adam optimizer, a learning rate of 0.001, and a batch size of 128. Depending on the dataset, training to convergence took 40–100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines Supporting contribution 3, we compare FLOWTIME against a broad and competitive set of baselines spanning three families: traditional statistical methods—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); deep learning methods—DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); diffusion models—TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021); and the flow-matching model TSFlow (Kolloviev et al., 2024), the most direct point of comparison for our approach.

Table 2: Evidence for contribution 3: test-set CRPS (lower is better) across five real-world datasets. FLOWTIME mean and standard error are obtained by re-training and evaluating five times; baseline results are taken from (Kolloviev et al., 2024). Best score per dataset in **bold**, second best underlined—FLOWTIME is best or second best on four of the five datasets.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 substantiates contribution 3 on five real-world datasets. FLOWTIME **sets a new state of the art on three of five datasets**—Electricity, Exchange, and Solar. It leads all baselines on Electricity with a CRPS of 0.042, a 6.7% improvement over the second-best TSFlow (0.045); it matches the best result on Exchange at 0.009, well ahead of the statistical and other deep learning methods; and, most strikingly, it cuts Solar CRPS to 0.284, a 17.2% reduction over the next-best TSFlow (0.343). **On the remaining two datasets it stays competitive:** it is second on Traffic (0.089, just behind TSFlow’s 0.083) yet still ahead of every traditional and deep learning baseline, and it reaches a competitive 0.243 on Wikipedia, trailing only WaveNet (0.207) and SSSD (0.209). The upshot is that autoregressive flow matching delivers superior or competitive performance across diverse real-world forecasting tasks—precisely the prior-free multivariate scalability our contributions promise.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive modeling with flow matching to produce accurate, well-calibrated predictions for multivariate timeseries. To recap the three contributions: (1) an autoregressive flow-matching formulation that factorizes forecasting into per-step conditional densities modeled by one shared flow, balancing the sequential structure of temporal data with the expressiveness of flow-based generative modeling; (2) prior-free scalability to high-dimensional multivariate data, removing the informed-prior restriction of earlier FM forecasters; and (3) an empirical study—an ablation on five dynamical systems plus comparisons on five real-world benchmarks—that validates these claims. Concretely, the autoregressive decomposition delivers notable gains in uncertainty calibration and extrapolation over standard flow-matching approaches that model entire future windows at once, and FLOWTIME generalizes robustly to out-of-distribution horizons, a capability that matters wherever test conditions diverge from training data.

These contributions come with limitations that scope their reach. First, the sequential nature of autoregressive sampling adds inference-time overhead relative to fully parallelizable non-autoregressive models, which may constrain real-time deployment. Second, our current implementation applies flow matching directly in the data space, which may be suboptimal for high-dimensional partially observed data. These openings point to concrete future work: latent-space formulations, extensions to irregularly sampled data, and broader applications in scientific modeling and decision-making under uncertainty. Overall, by uniting an autoregressive factorization with flow matching,

FLOWTIME is a step forward in probabilistic timeseries forecasting, showing that flow-based generative models can improve accuracy and uncertainty quantification together.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.

486 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
487 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
488 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
489 28364, 2023.

490 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
491 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
492 *International Conference on Learning Representations*, 2024.

493 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
494 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

495 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
496 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
497 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

500 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
501 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

502 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
503 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

504 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
505 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
506 *tions*, 2022.

507 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
508 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

509 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

510 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

511 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
512 *preprint arXiv:2401.03006*, 2024.

513 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
514 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
515 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

516 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
517 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
518 *arXiv:2002.06103*, 2020.

519 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
520 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
521 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

522 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
523 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
524 *in Neural Information Processing Systems*, 32, 2019.

525 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
526 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
527 1181–1191, 2020.

528 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
529 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
530 *ing*, pp. 2256–2265. PMLR, 2015.

531 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
532 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
533 *arXiv:2011.13456*, 2020.

540 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
541 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
542 *ative Modeling*, 2024.

543 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
544 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
545 *cessing Systems*, 34:24804–24816, 2021.

546 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-
547 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
548 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

549 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
550 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
551 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

552 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
553 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

554 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
555 neural networks. *Neural Computation*, 1(2):270–280, 1989.

556 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
557 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
558 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

559 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
560 series foundation models: Generalizing time series representation with large language model. In
561 *ArXiv:2104.00164*, 2024.

562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593

A DATASETS

A.1 DYNAMICAL SYSTEMS

To construct the controlled ablation that underpins contribution 1, we generated training and testing data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their corresponding stochastic differential equations (SDEs). Each system has the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where, f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. For each system we sampled 2400 random initial conditions from a uniform distribution over the specified range (see Table 3) and generated one trajectory per initial condition by solving the SDE with the Euler-Heun method at fixed step size, recording 200 equally spaced time points. Each trajectory is split into the first 75 points as observed data, the next 75 for prediction, and the final 50 for extrapolation—the split that lets Table 1 report prediction and extrapolation separately. Of the 2400 trajectories, 2000 are used for training and 400 for testing.

Table 3: Specification of the five stochastic dynamical systems used in the contribution 1 ablation. $\mathbf{f}(\mathbf{x})$ is the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial-condition distribution. Each system was integrated over the interval $[t_0, t_1]$ with 200 steps using the Euler-Heun method.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra		$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\beta = 0.9$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

To evaluate the prior-free multivariate scalability of contribution 2, we used five datasets from GluonTS (Alexandrov et al., 2020), in the versions preprocessed as in (Salinas et al., 2019). Their properties are summarized in Table 4.

Table 4: Properties of the real-world datasets used to demonstrate prior-free multivariate scalability (contribution 2): dimension n , domain Ω , frequency, total training timesteps, and prediction length. Note the dimensionality n ranging up to 2000, illustrating the high-dimensional regime FLOWTIME handles without informed priors.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting This appendix fully specifies the non-autoregressive baseline that serves as the comparison point isolating contribution 1. We keep the forecasting setup of Section 3—given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is still to model $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ —but, in contrast to the autoregressive factorization, we model the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ directly, without any temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps but pays for it by having to learn a high-dimensional distribution—the difficulty our autoregressive factorization is designed to avoid.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes per dimension, where Brownian motion is defined as a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that operates on the entire future trajectory. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

using numerical solvers. This produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Implementation Details We ran this baseline on the dynamical-systems forecasting task that anchors the contribution 1 ablation. Here ν_θ is a 4-layer bi-directional LSTM with 128 hidden units, and all other training parameters match Section 4.1 so that the comparison isolates the factorization. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant Laying out these differences clarifies exactly what contribution 1 buys and where the ablation draws the line. First, on factorization: the autoregressive method decomposes the joint distribution Markovianly across time steps, whereas the non-autoregressive baseline models the full trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable—preserving cross-temporal dependencies at the cost of learning a far more complex distribution over $\mathbb{R}^{n \times f}$. Second, on computation: the non-autoregressive version pushes entire trajectories through sequential velocity-field networks, enabling parallel generation of all time-points but demanding more memory. Third, on uncertainty: the autoregressive approach explicitly models how prediction errors accumulate through the chain of conditional distributions, while the non-autoregressive method captures joint uncertainty via trajectory-level sampling but has no

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

```
1: while not converged do
2:   Sample  $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$  and  $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$ 
3:   Set  $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$ 
4:   Sample  $s \sim \mathcal{U}(0, 1)$ 
5:    $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$ 
6:   Compute target  $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$ 
7:   Update  $\theta, \phi$  to minimize  $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$ 
8: end while
```

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

```
1: Encode context:  $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$ 
2: Sample initial trajectory  $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ 
3: Solve  $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$ 
4: Return  $\psi(\mathbf{Y}_f, 1)$ 
```

explicit mechanism for error-accumulation dynamics. The net picture is one of complementary trade-offs—the autoregressive variant of FLOWTIME offers interpretable uncertainty and efficient window-based computation under a limited Markovian assumption, while the non-autoregressive baseline retains full temporal correlations at higher computational cost and with less explicit uncertainty dynamics—and it is precisely this contrast that the dynamical-systems ablation quantifies.

C EVALUATION METRICS

To make the empirical claims of contribution 3 precise, we detail the two metrics used throughout. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and its predictive distribution or samples from our probabilistic model, we report the Continuous Ranked Probability Score (CRPS), which targets calibration, and the Normalized Root Mean Squared Error (NRMSE), which targets point accuracy.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule commonly used to assess the quality of probabilistic forecasts, and it is the primary metric behind our calibration claims. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS measures how well the entire predictive distribution aligns with the observation x ; lower values indicate better-calibrated and more accurate predictive distributions—hence its role in ranking methods in Table 2.

For time series forecasting, we compute the CRPS at each time step in the horizon and average:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF (or an empirical CDF from samples) at forecast horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard metric for point-forecast accuracy, and it underlies the accuracy comparisons in our dynamical-systems ablation. For a set of predictions

$\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make the RMSE scale-invariant and comparable across datasets or series of differing magnitudes, we use the Normalized RMSE (NRMSE), normalizing by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations; alternatively one could normalize by the data range ($\max(y) - \min(y)$), depending on the application. A lower NRMSE indicates more accurate predictions relative to the variability of the series, which is how we read the accuracy gains reported for FLOWTIME.